

Submission Coursework Brief

School of Computing and Engineering

Unit: PROGRAMMING (COMP4015)		
Unit Contact: Ismail Alarab	Credits: 20	Level: 4

Assessment Title: Programming with Lab Exercises and Application Project	
Assessment Number: 1 of 1	
Assessment Type: Individual	Weighting: 50%
Deadline: 09/12/2025 12:00 PM	Submission Method: Large File Submission Box

Quality Assessor (QA): Botao Fan	Other Marker(s): Hari Pandey
---	-------------------------------------

Can I use Generative AI tools?

Basic spelling and grammar correction tools are permitted.

Generative AI task completion

Generative AI can be used to complete specified elements of the coursework as detailed below. Any generative AI content must be acknowledged. Any elements not specified must be completed without the use of generative AI.

Tasks:

This coursework has two programming tasks (Task A and Task B) and one report. You must complete all three components to receive marks.

Task A - Assessed Lab Exercises:

- During the first eight weeks, some of lab exercises are labelled ASSESSED TASKS on Brightspace. You must complete these exercises using Python functions. Each function should:
 - Solve the problem as described.
 - Validate input and output.
 - Be tested to confirm correct behaviour.
- Do not submit weekly. Instead, collect all the assessed exercises into a single portfolio file named **assessed_lab_exercises.py** and submit it once, with the rest of your coursework, at the final deadline.

Task B - Vending Machine:

Write a console-based Python program that simulates a vending machine.

Requirements:

- Accepts coins £1, £2, 50p, 20p.
- Contain at least five items with prices and stock levels (e.g., water £1.20, soda £1.50, chocolate £2.50, crisps £1.40, sandwich £3.80).
- Allow purchases if funds and stock are sufficient. Deduct cost from user's balance.
- Return change using only multiples of the accepted coins; any remaining amount that cannot be returned is kept.
- Handle invalid inputs, insufficient funds, and out-of-stock items.
- Apply discounts:
 - If you buy 3 items → 5% discount
 - If you spend more than £5 → 10% discount
 - If you spend more than £7 → 15% discount
 - If more than one discount applies, use the highest discount only, rounded down.
- Bonus rule: every odd-numbered order (1st, 3rd, 5th, ...) who spends more than £2 receives £1 extra change, regardless of any discounts applied.
- The program must:
 - Use functions and loops for modularity.
 - Maintain accurate state across purchases (balance, stock, discounts).
 - Be interactive, clear, and user-friendly.
 - Include a feature to generate a CSV receipt when an order is completed, recording order number, purchased items, costs, discounts, total spent, and remaining balance, then continue to the next customer.
- The figure below shows a snippet of the game's console interaction as an example:

```
▼ TERMINAL Python + ▢ 🗑️ ...
Current balance: £0.00
Insert coin (£2, £1, 50p) or type 'done' to finish: 2
Current balance: £2.00
Insert coin (£2, £1, 50p) or type 'done' to finish: 2
Current balance: £4.00
Insert coin (£2, £1, 50p) or type 'done' to finish: done

--- Vending Machine Menu ---
1. Water - £1.5 (10 in stock)
2. Soda - £1.5 (10 in stock)
3. Chocolate - £2.5 (5 in stock)
4. Crisps - £1.5 (8 in stock)
5. Sandwich - £3.5 (5 in stock)
-----
Current balance: £4.00
Select an item by name or type 'exit' to quit: water
Purchased water for £1.50.
Total spent: £1.50 (Discount applied: £0)
Remaining balance: £2.50

--- Vending Machine Menu ---
1. Water - £1.5 (9 in stock)
2. Soda - £1.5 (10 in stock)
3. Chocolate - £2.5 (5 in stock)
4. Crisps - £1.5 (8 in stock)
5. Sandwich - £3.5 (5 in stock)
-----
Current balance: £2.50
Select an item by name or type 'exit' to quit: █
```

Report: Write a report (max **650 words**, excluding figures, references, and appendices). It should include:

1. **Task A:** Provide evidence of testing your functions, such as screenshots of your unit tests and their results.
2. **Task B:**
 - Include a sketch of your program's design (e.g., paper drawing, flowchart, pseudocode, or diagram).
 - Show evidence of progress with screenshots of your program at different implementation stages.
 - Explain how you tested your program, covering both normal and edge-case scenarios, supported by screenshots of test runs.
3. **Reflection:** Summarise your learning, challenges faced, and improvements made during development.
4. **AI Use** (if applicable):
 - Reference the tool used in BU APA format.
 - Include a link or screenshot of your prompts and the AI's responses.
 - Briefly explain how any AI-generated content was adapted in your final work.

Intended Learning Outcomes (ILOs)

1. Design and implement solutions to different problems using appropriate programming languages.
2. Demonstrate the appropriate use of basic data-structures in software programs.
3. Demonstrate an understanding of the importance of developing software in an ethical, secure, and professional manner.
4. Test and debug a given program by using a suitable testing strategy.

Submission Format:

Before the submission deadline, submit the following to Brightspace:

- A ZIP file containing two Python files: (**assessed_lab_exercises.py** , **vending_machine.py**).
- Report (PDF – max 650 words).

A report submitted without the code will be treated as a non-submission.

How will this be assessed?

Criterion 1 - Task A: (30%)

- All assessed lab exercises completed and functional.
- Functions correctly validate and test input/output.
- Good coding practices: naming, comments, layout, consistent style.

Criterion 2 - Task B: (30%)

- Vending machine program runs correctly, meets requirements, and maintains state.
- Effective use of data types, structures, and algorithms.
- Clear, readable, well-commented code with no unresolved warnings/errors.

Criterion 3 – Professionalism and Report: (40%)

- Task A: Include screenshots of unit tests and results.
- Task B:
 - Design & Development: Provide a program sketch (paper drawing, flowchart, pseudocode, or diagram, etc.) and screenshots of evidence of progress.
 - Testing: Show normal and edge-case testing with screenshots.

- Reflection: Summarise learning, challenges, and improvements.
- AI Use (if applicable): Declare AI assistance using BU APA references and screenshots/links; explain how outputs were adapted.
- Presentation: Ensure clarity, organisation, and professional formatting.

Total = 100%

Questions about this assessment:

Questions about the assignment brief can be asked to the teaching team by email or Microsoft Teams. Please ask us any questions you wish – if you are at the beginning and not sure where to start – that is a valid question – please ask us and we can help you.

Academic Integrity

The work you submit must be your own. Any attempt to gain an unfair advantage in your assessment by **cheating**, deception or fraud is considered an academic offence. The 'Assessment help and support' section of the unit (found under 'Assessment' in the content area) provides more guidance on avoiding academic offences, including **any guidance on what will or will not be considered an academic offence in this specific assessment.**

Help and support

The 'Assessment help and support' section of the unit (found under 'Assessment' in the content area) provides information and guidance, including specific information on support for this assessment. It provides help with our policies on deadline extensions and information on support available in the university, including academic skills support and additional learning support for students with disabilities.

Disclaimer: The information provided in this assignment brief is correct at time of publication. In the unlikely event that any changes are deemed necessary, they will be communicated clearly via e-mail and via the VLE and a new version of this coursework brief will be circulated.

Version: 4

Updated: October 2025

Issued Date: 11/09/2025

Guide to Levels of Attainment (Appendix A)

	Tasks Completed and Understood	Quality of Code	Professional Manner
Criteria examined	Tasks completed successfully i.e.: functional fulfilling task requirements and passing the tests where necessary	Well selected data types, structures and algorithms using appropriate methods. Layout and whitespace applied appropriately Naming conventions and meaningful names used Warnings eliminated	Report demonstrates completed code for Tasks A and B. Briefly answers Task A and B questions with unit test examples and systematic testing strategies. Includes appropriate reflection on learning and use of AI. Submitted in correct format.
Excellent (> 80)	ALL or nearly all ... tasks completed. Perhaps one is functional but not working as expected. High level of understanding demonstrated	No improvement (or very little) needed Selection of data types, structures algorithms and methods are sound on the whole. Regular insightful commenting. Good layout Good naming of variables. No warnings	No/Very little improvement needed Report clearly demonstrates code for Tasks A & B and thoroughly answers both questions with unit test snippets and systematic testing strategies, including normal and edge cases with insightful reflection on learning and clearly use of AI. Submitted in correct format.
Good (65 – 80)	MOST... Most tasks completed Good level of understanding	Little improvement needed Selection of data types, structures algorithms and methods occasionally not best choice Regular commenting. Occasional layout error Good naming of variables – occasional error One warning	Little improvement needed. Report demonstrates code for most tasks and adequately answers both questions with some unit test or testing strategy examples, includes some reflection on learning and mentions AI use.
Average (50 – 65)	SOME ... Approx half tasks completed or perhaps 1) not much of the timed assessment and the lab tasks or 2) only the timed assessment completed. Occasional gaps in understanding	Some improvement needed Selection of data types, structures algorithms and methods considered although some not best choice. Layout mostly good. Occasionally naming variables lacks meaning/correct format. Some warnings dealt with.	Some improvement needed. Report shows code for some tasks and partially answers questions; some unit test or testing strategy examples missing or incomplete, reflection on learning or AI use is limited.
Weak (40 – 50)	A FEW... Two tasks work as expected. Other code incomplete/does not compile Many gaps in understanding	Much improvement needed Selection of data types, structures algorithms and methods often not best choice Occasional comments. Poor layout in some places. Occasional mistakes in variable name format A few warnings dealt with.	Much improvement needed Report demonstrates minimal code and superficially answers questions; few or poor examples of unit tests or testing strategies; reflection and AI use poorly addressed.
Failing (< 40)	NONE or VERY FEW... tasks completed /does not compile Little/no evidence of understanding	A lot of work needed to pass Poor selection of data types, structures algorithms and methods, if they are used. Poor naming of variables. Warnings wherever they normally occur still in place. No comments. Poor layout.	A lot of work needed to pass Report missing or fails to address questions; no examples of unit tests or testing strategies; little or no reflection and no mention of AI use. Not submitted in a correct format.